

Universidad Politécnica de Madrid

Escuela Técnica Superior de Ingenieros Informáticos



Arquitectura de Coche compartido: Practica 00

2º entrega

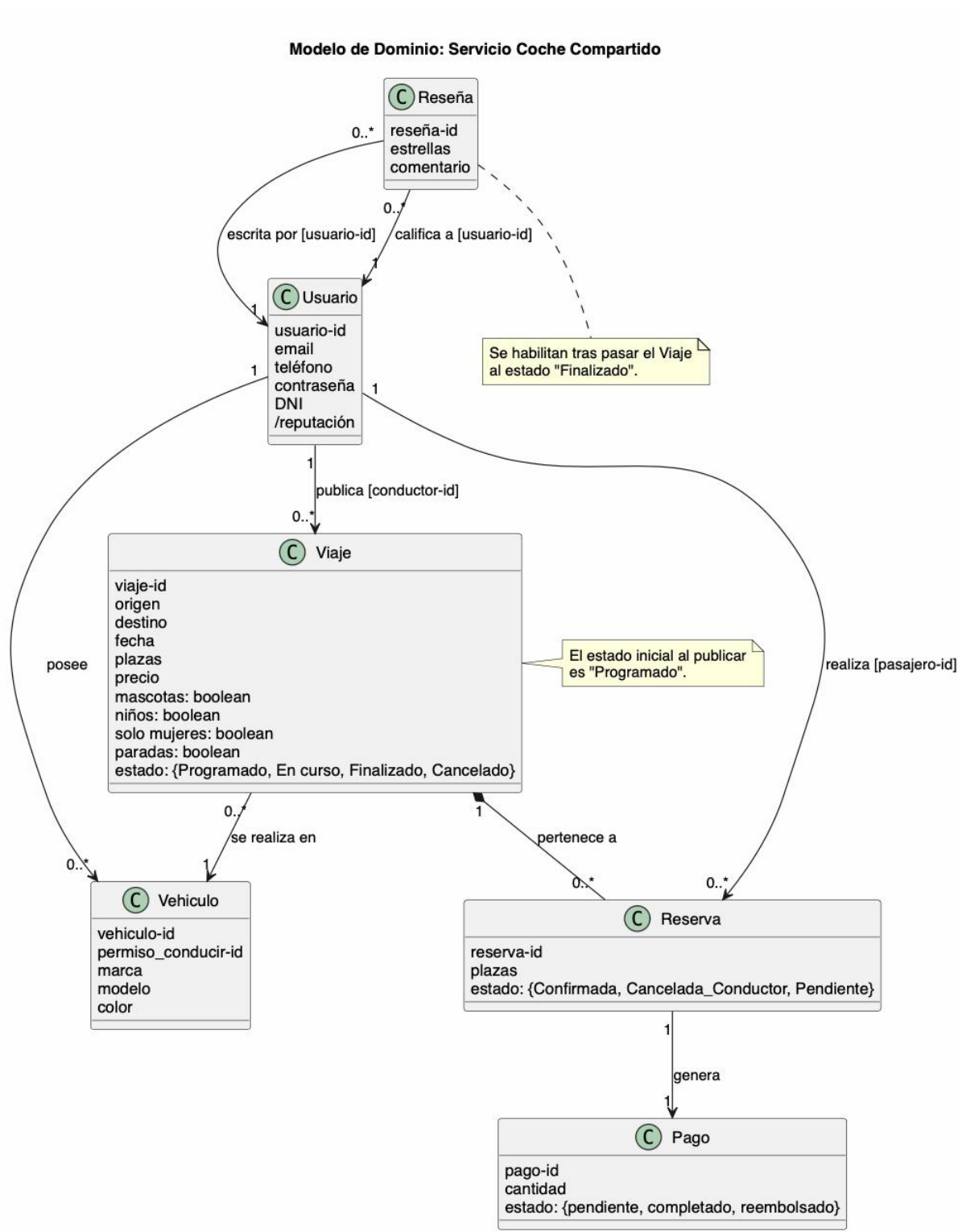
Autores: Rubén Jaraba Sánchez, Clara García Toledo, Pablo Campo Herrero,
Alexandra Pinto Ghaffar

Contenido

Modelo de dominio.....	4
Caso de Uso de Reserva de un viaje.....	5
Diagrama de secuencia	5
Justificación de patrones GRASP	5
Caso de Uso de Publicación de viaje con sincronización CQRS	8
Diagrama de secuencia	8
Justificación de patrones GRASP	8
Caso de Uso de Cancelación del trayecto.....	12
Diagrama de secuencia	12
Justificación de patrones GRASP	12
Caso de Uso de Finalización del trayecto.....	15
Diagrama de secuencia	15
Justificación de patrones GRASP	15
Fotos diagramas de secuencia ampliadas	18
Reserva de un viaje	18
Publicación de viaje con sincronización CQRS	19
Cancelación del trayecto.....	20
Finalización del trayecto	21

Modelo de dominio

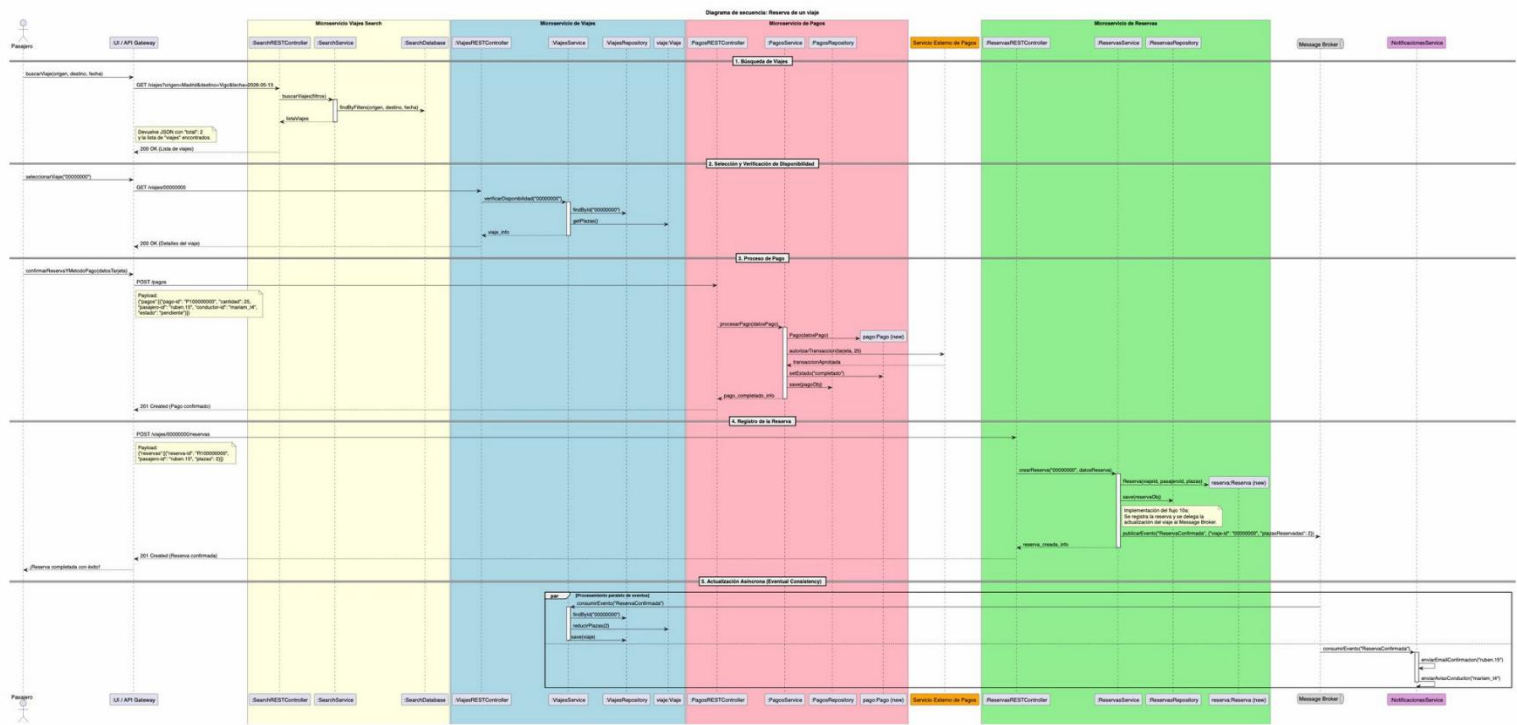
Representamos el modelo de dominio a través de la siguiente imagen, tomando en cuenta todos los casos de uso.



Caso de Uso de Reserva de un viaje

Diagrama de secuencia

Representación del diagrama



Al final del documento aparecen las fotografías ampliadas para su mejor comprensión

Justificación de patrones GRASP

Patrón GRASP	¿Se aplica? Sí/No	Caso de uso	Ejemplos (máximo 2). Indicar objetos involucrados.	Justificación breve
Experto en información	Sí	Reserva de viaje	La responsabilidad de saber si hay plazas libres recae en viaje: Viaje mediante el método getPlazasLibres()	La clase Viaje posee el atributo plazas y las relaciones con las reservas existentes. Por lo que, Viaje posee la información necesaria para determinar el número de asientos disponibles.
Creador	Sí		El objeto ReservasService es	ReservasService recibe a través del controlador

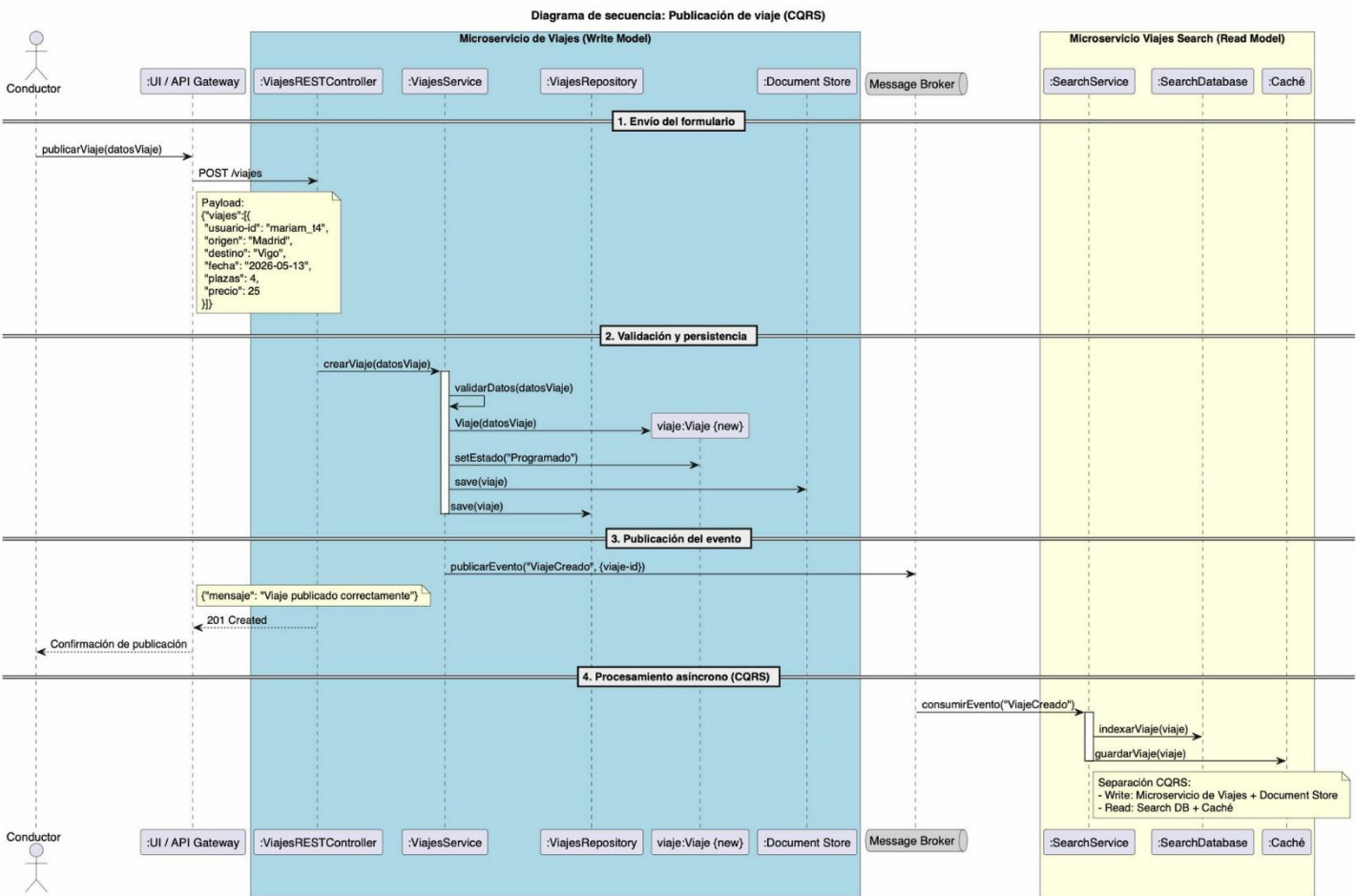
Patrón GRASP	¿Se aplica? Sí/No	Caso de uso	Ejemplos (máximo 2). Indicar objetos involucrados.	Justificación breve
		Reserva de viaje	responsable de instanciar la nueva reserva.	todos los datos necesarios provenientes del usuario para inicializar correctamente el objeto Reserva
Controlador	Sí	Reserva de viaje	El uso de las clases SearchRestController, ViajesRestController, PagosRestController y ReservasRestController actuando como interfaz de entrada	Estas clases interceptan las peticiones que provienen del Pasajero y actúan como controladores de fachada para sus respectivos microservicios separando así la capa de presentación/comunicación de la lógica de negocio
Bajo acoplamiento	Sí	Reserva de viaje	El uso de un Message Broker	En lugar de que el ReservasService llame directamente al ViajesService para reducir una plaza y al NotificacionesService para enviar un email, simplemente publica un evento. Esto significa que el microservicio de Reservas no necesita conocer los detalles de los otros servicios
Alta cohesión	Sí	Reserva de viaje	El comportamiento del ReservasService	ReservasService solo se encarga de instanciar y guardar la reserva. Al delegar el resto de las tareas a otros componentes específicos, se evita crear una clase gigante que haga todo, logrando que cada servicio sea fácil de entender y mantener
Polimorfismo	No	Reserva de viaje		
Fabricación pura	Sí	Reserva de viaje	Las clases como SearchService, ViajesRepository, ReservasRepository y el Message Broker	Estas clases no existen en el modelo de dominio, se han creado para la persistencia de datos, evitando que otras clases tengan lógica de base de datos.

Patrón GRASP	¿Se aplica? Sí/No	Caso de uso	Ejemplos (máximo 2). Indicar objetos involucrados.	Justificación breve
Indirección	No	Reserva de viaje		
Variaciones protegidas	No	Reserva de viaje		

Caso de Uso de Publicación de viaje con sincronización CQRS

Diagrama de secuencia

Representación del diagrama



Al final del documento aparecen las fotografías ampliadas para su mejor comprensión

Justificación de patrones GRASP

Patrón GRASP	¿Se aplica? Sí/No	Caso de uso	Ejemplos (máximo 2). Indicar objetos involucrados.	Justificación breve
Experto en información	si	Publicación de viaje	ViajesService es quien tiene acceso a los datos del viaje entrante y al repositorio, por lo que es el experto para validar y decidir si el viaje es correcto.	Se asigna la responsabilidad de validación a la clase que tiene toda la información necesaria para poder hacerla.
Creador	si	Publicación de viaje	ViajesService crea la instancia viaje:Viaje mediante new Viaje(datosViaje).	Se asigna la creación del objeto a la clase que posee los datos necesarios para inicializarlo, evitando que esa responsabilidad recaiga en el controlador o en la capa de presentación.
Controlador	si	Publicación de viaje	ViajesRestController recibe el evento del sistema POST /viajes desde el API Gateway y lo delega a ViajesService.	Se separa la recepción del evento del sistema de su procesamiento. ViajesRestController actúa como controlador de fachada en el caso en el que se publique algo.
Bajo acoplamiento	si	Publicación de viaje	ViajesService no conoce ni llama directamente a	La comunicación asíncrona a través del Message Broker

Patrón GRASP	¿Se aplica? Sí/No	Caso de uso	Ejemplos (máximo 2). Indicar objetos involucrados.	Justificación breve
			SearchService. Solo publica el evento "ViajeCreado" en el Message Broker. SearchService consume ese evento de forma independiente.	elimina el acoplamiento directo entre el lado de escritura y el lado de lectura, permitiendo que ambos evolucionen independientemente.
Alta cohesión	si	Publicación de viaje	El SearchService tiene una única responsabilidad: mantener el índice de búsqueda actualizado. Solo indexa en SearchDatabase e invalida la Caché. No mezcla lógica de negocio de viajes, pagos ni reservas.	Separar el lado de lectura (SearchService) del lado de escritura (ViajesService) siguiendo CQRS garantiza que cada clase tenga responsabilidades altamente cohesionadas y enfocadas.
Polimorfismo				
Fabricación pura	si	Publicación de viaje	ViajesRepository y SearchDatabase no representan conceptos del dominio del problema (no como Viaje o Usuario), sino clases artificiales creadas para gestionar la persistencia y la indexación, manteniendo la cohesión de	Se introduce una clase de conveniencia para aislar responsabilidad en otras clases, evitando que las clases de dominio se acoplen directamente a la lógica del almacenamiento.

Patrón GRASP	¿Se aplica? Sí/No	Caso de uso	Ejemplos (máximo 2). Indicar objetos involucrados.	Justificación breve
			ViajesService y SearchService.	
Indirección	si	Publicación de viaje	El Message Broker actúa como intermediario entre ViajesService y SearchService. Ninguno de los dos se conoce directamente; toda la comunicación pasa por el broker.	Se introduce un objeto intermediario (Message Broker) para desacoplar el productor del evento del consumidor, permitiendo que el lado de escritura y el de lectura sean independientes.
Variaciones protegidas	si	Publicación de viaje	SearchService encapsula la SearchDatabase y la Caché. El resto del sistema nunca accede directamente a ellas solo interactúa con SearchService a través de eventos. Si se cambia el motor de búsqueda o la estrategia de caché, el resto del sistema no se ve afectado.	Se protege el sistema frente a cambios en la tecnología de búsqueda o caché creando una interfaz estable (SearchService) que absorbe esas variaciones internas.

Patrón GRASP	¿Se aplica? Sí/No	Caso de uso	Ejemplos (máximo 2). Indicar objetos involucrados.	Justificación breve
Experto en información	Sí	Anular reserva	El objeto reservasController llama a cambiarEstado() sobre el objeto reservaActual. El objeto viajesController llama a	Tanto reservaActual como viajeAsociado guardan su estado y sus plazas. Además también son los encargados de actualizar sus valores

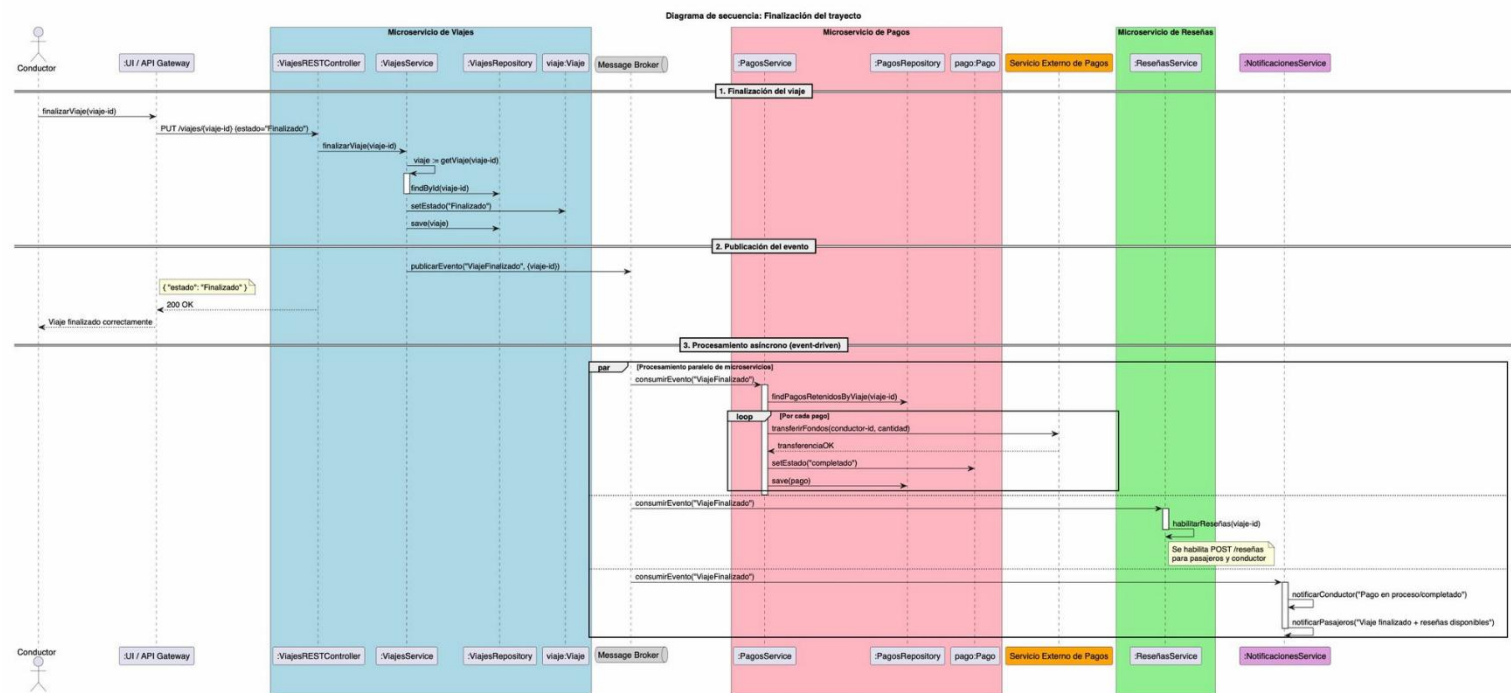
Patrón GRASP	¿Se aplica? Sí/ No	Caso de uso	Ejemplos (máximo 2). Indicar objetos involucrados.	Justificación breve
			incrementarPlazasDisponibles() sobre el objeto viajeAsociado.	
Creador	No	Anular reserva		Durante la anulación de una reserva, no se crean nuevas entidades de dominio, solo se modifican los estados de los objetos que ya existen.
Controlador	Sí	Anular reserva	El objeto reservasController.	Es el primer objeto (sin contar con la capa externa) que recibe y coordina la petición de anular reserva.
Bajo acoplamiento	Sí	Anular reserva	El uso de MessageBroker por parte del reservasController.	Como publica un evento asíncrono, el controlador de reservas no necesita conocer la existencia ni de pagos ni de viajes, manteniendo el acoplamiento al mínimo.
Alta cohesión	Sí	Anular reserva	La división de tareas entre reservasController, pagosController y viajesController.	Cada controlador está muy bien definido en su campo y diferenciado del resto, así que se centra solo en su propio dominio (gestionar la reserva, procesar el reembolso o actualizar el

Patrón GRASP	¿Se aplica? Sí/ No	Caso de uso	Ejemplos (máximo 2). Indicar objetos involucrados.	Justificación breve
				viaje), lo que permite una alta cohesión entre todas las partes.
Polimorfismo	Sí	Anular reserva	El objeto adaptadorPagos (PasarelaPagosAdapter).	El controlador de pagos interactúa con la interfaz, sin saber cómo funciona cada pasarela bancaria.
Fabricación pura	Sí	Anular reserva	El MessageBroker.	No representa un concepto real del dominio del negocio. Se crea artificialmente para permitir la mensajería asíncrona.
Indirección	Sí	Anular reserva	El Message broker.	Es un intermediario entre reservasController y los controladores pagosController o viajesController.
Variaciones protegidas	Sí	Anular reserva	El objeto adaptadorPagos encapsulando la API externa.	Protege el núcleo del sistema (pagosController) frente a posibles cambios en la API del servicio externo de pagos. Si el banco cambia sus reglas, solo se modifica el adaptador.

Caso de Uso de Finalización del trayecto

Diagrama de secuencia

Representación del diagrama



Al final del documento aparecen las fotografías ampliadas para su mejor comprensión

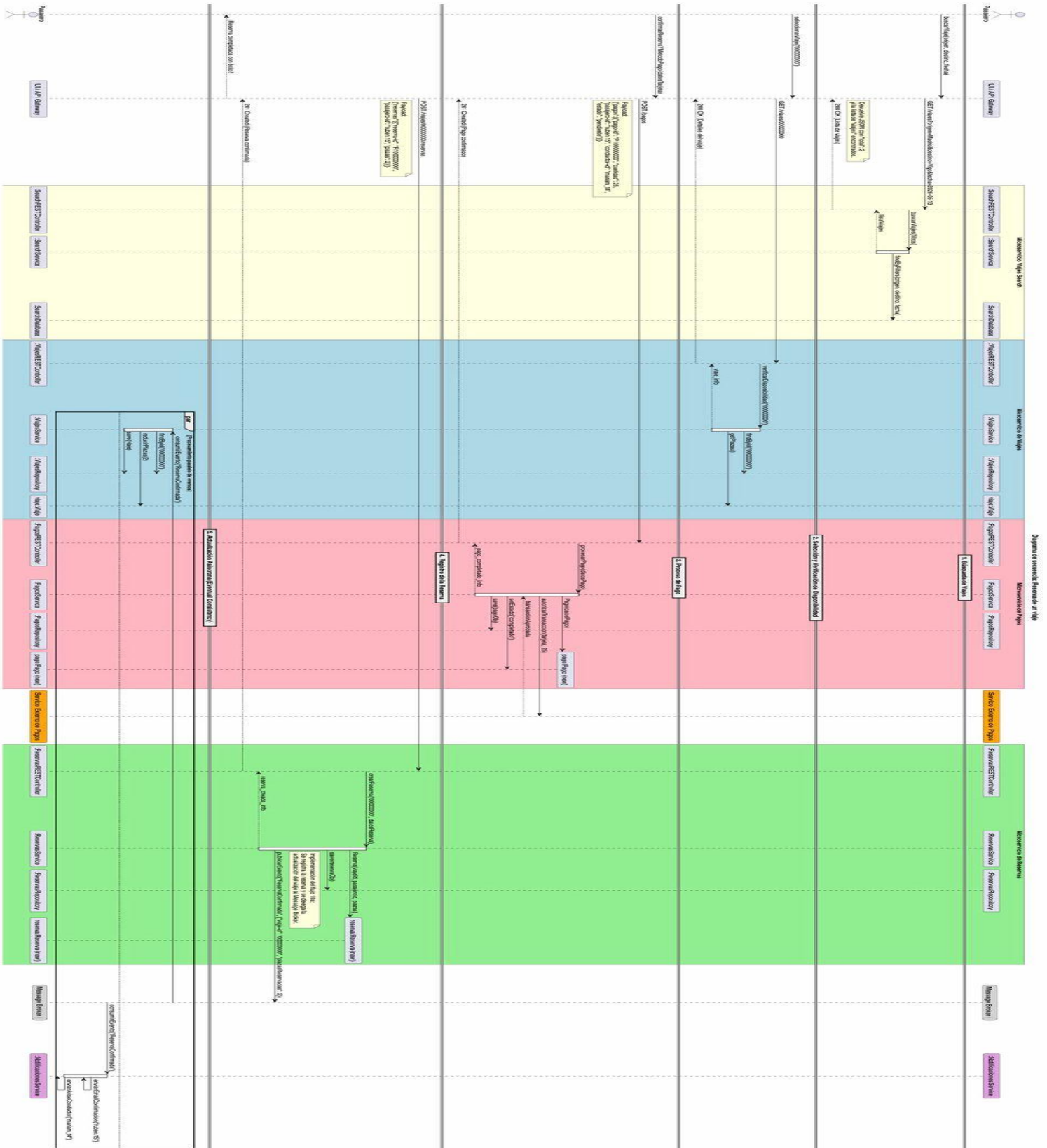
Justificación de patrones GRASP

Patrón GRASP	¿Se aplica? Sí/ No	Caso de uso	Ejemplos (máximo 2). Indicar objetos involucrados.	Justificación breve
Experto en información	Sí	Finalización del trayecto	viajesController llama a cambiarEstado() sobre el objeto viajeActual.	El objeto viajeActual es el que posee la información de su propio estado, por lo que es el responsable de modificarlo.

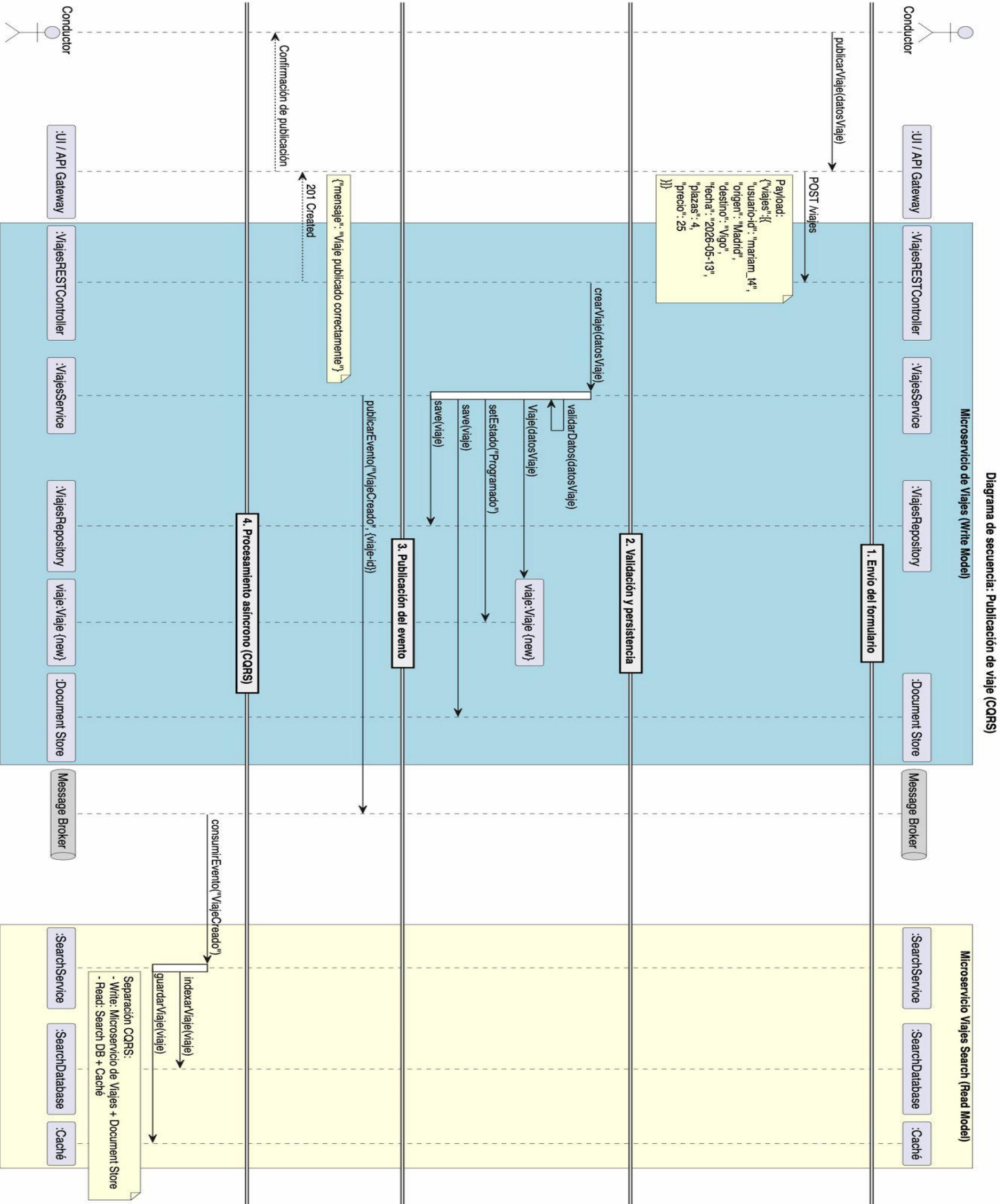
Patrón GRASP	¿Se aplica? Sí/ No	Caso de uso	Ejemplos (máximo 2). Indicar objetos involucrados.	Justificación breve
Creador	No	Finalización del trayecto		En el flujo básico de finalizar un viaje se actualizan estados existentes, pero no se crean nuevos objetos de dominio directamente.
Controlador	Sí	Finalización del trayecto	El objeto viajesController.	Actúa como controlador de caso de uso, recibiendo la petición de la capa externa y coordinando la actualización del viaje.
Bajo acoplamiento	Sí	Finalización del trayecto	Uso de MessageBroker entre viajesController y pagosController.	viajesController no necesita conocer la existencia del resto de objetos (pagos o reseñas), solo publica un evento, reduciendo en gran medida el acoplamiento.
Alta cohesión	Sí	Finalización del trayecto	División en viajesController, pagosController y reseñasController.	Cada controlador tiene una responsabilidad única, separada del resto y enfocada en su dominio.
Polimorfismo	Sí	Finalización del trayecto	El objeto PasarelaPagosAdapter.	El adaptador implementa una interfaz de pagos para que el

Patrón GRASP	¿Se aplica? Sí/ No	Caso de uso	Ejemplos (máximo 2). Indicar objetos involucrados.	Justificación breve
				controlador no dependa de un banco específico.
Fabricación pura	Sí	Finalización del trayecto	Los objetos Message broker y notificador.	No representan conceptos del negocio de viajes compartidos. Se han inventado artificialmente para manejar correctamente la mensajería y los correos.
Indirección	Sí	Finalización del trayecto	El objeto MessageBroker.	Es un intermedio entre la finalización del viaje y el cobro y las reseñas.
Variaciones protegidas	Sí	Finalización del trayecto	Uso de adaptadorPagos.	Protege a pagosController de los cambios en la API del Servicio Externo, sin grandes cambios.

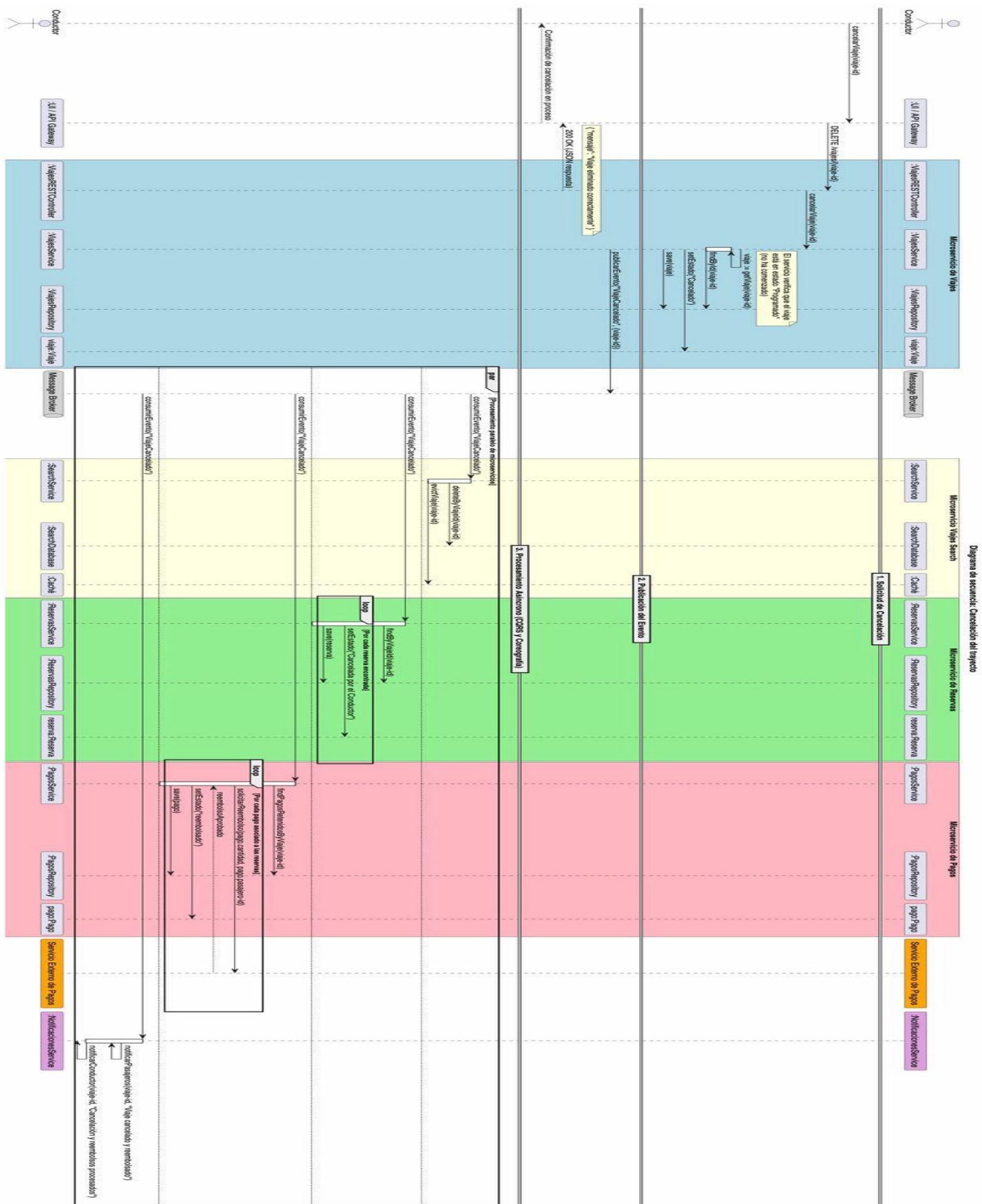
Reserva de un viaje



Publicación de viaje con sincronización CQRS



Cancelación del trayecto



Finalización del trayecto

